

TREES AND THEIR DEGREES

MARIE-CAMILLE DELARUE

CONTENTS

1. Introduction	1
2. A question about trees	2
2.1. Trees with simple junctions	2
2.2. Trees with double junctions	2
2.3. Goal – notion of degree	3
3. Defining the detector map	4
4. Degree of a map between spheres	4
4.1. Degree formula	5
4.2. Algorithm	6
5. Results	7
6. Limitations	7
6.1. Limitations of the degree computation	7
6.2. Other problems	7

1. INTRODUCTION

In the course of our study of the family of Higman–Thompson groups, there arose a question that was easy to answer in small dimensions and much harder in higher dimensions. The fundamental reason for this is that it is much harder to obtain a reasonable visualization in higher dimensions. The following document explains this problem and the framework we came up with to answer it in higher dimensions. Reading the following document is recommended to understand the specific question we were working on as well as its implementation.

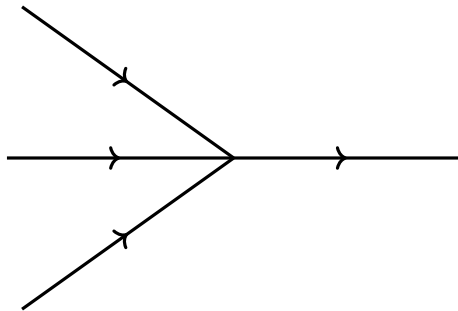
Topological spaces are easier to understand when broken up into simpler spaces with maps between them. Because the Higman–Thompson groups can be represented as groups of trees, we wanted to understand the topology of spaces of trees. In this case, there is one building block per type of junction in the trees, and the maps between the building blocks are determined by how the junctions attach to each other.

Date: September 14, 2026.

2. A QUESTION ABOUT TREES

We will consider finite rooted trees, and we call an **a -junction** a vertex of the tree with one parent and a children: For instance, Υ is a 3-junction. We first consider the case of trees with simple junctions.

2.1. Trees with simple junctions. Consider a finite rooted tree which only has a -junctions, for a an integer. We will think of trees as being oriented from the leaves to the root. The following is an illustration of an (oriented) junction:



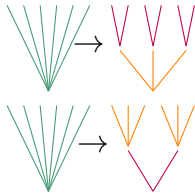
We will call **degree** of a junction the number of incoming edges minus the number of outgoing edges at the junction. For any a -junction, this number is clearly $a - 1$. If the trees have two or more types of junctions, for instance a -junctions and b -junctions, the a -junctions have degree $a - 1$, and the b -junctions have degree $b - 1$.

2.2. Trees with double junctions. Now let us consider a family of trees with both a -junctions and b -junctions, for a and b two different integers, with the following additional **interchange relation**:

$$\begin{array}{c} \text{V} \quad \text{V} \quad \text{V} \\ \diagdown \quad \diagup \quad \diagdown \\ \text{V} \end{array} = \begin{array}{c} \text{V} \quad \text{V} \quad \text{V} \\ \diagdown \quad \diagup \quad \diagup \\ \text{V} \end{array} .$$

This means that we identify certain trees even if they do not have the same shape.

We are interested in embedding these trees into an N -dimensional cube, thereby going from a combinatorial tree to a geometric tree. How does the equality between the combinatorial trees translate to the geometric context? In topology, we can always relax an equality between two objects by creating a path between them. We add an ab -junction which represents this path:



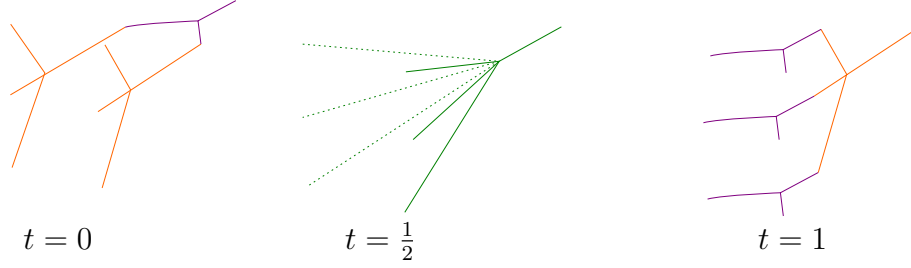


FIGURE 1. The tree is parametrized by $(x, t) \in [0, 1]^N \times [0, 1]$. The barycenter of the junctions is governed by x and the shape of the tree (including how far apart the junctions are) is governed by t .

If our tree lives in N -dimensional space, we are adding an extra dimension which represents time. The position of t in $[0, 1]$ indicates what the form of the tree is in a continuous way as depicted in figure 1. Now our tree is parametrized by an element in $[0, 1]^{N+1}$.

In the simple junction case and in the double junction case, there is a function `make_combinatorial_tree` which holds the combinatorial information about the tree, and a function `make_geometric_tree_at` which to a point in $[0, 1]^{N+1}$ and the combinatorial information of the tree associates the corresponding embedding of a tree inside a unit cube $[0, 1]^N$.

2.3. Goal – notion of degree. We would like to determine the degree of this extra junction. It is tempting to say that the degree of this junction is $ab - 1$ because of the middle picture in figure 1. However, the equivalent trees at $t = 0$ and $t = 1$ have a very different structure, and it is not immediately clear why their degree should be $ab - 1$.

Let's give another definition of the degree by going back to the case of a simple junction. We represent the junction living inside a small neighborhood, depicted as a circle around it in figure 2. Then imagine a smaller lens traveling around the boundary of the small neighborhood and recording what it sees. The lens will see a single path $a + 1$ times: a time with positive orientation and once with negative orientation, and so we recover a total degree of $a - 1$.

We will find the degree of the ab -junction by traveling around it and recording what we see. We need to count the number of a -junctions and b -junctions that we see around it. This is how we will construct the detector map.

3. DEFINING THE DETECTOR MAP

In the simple junction case, the detector takes the embedding of an a -junction inside a cube at a point p and records the position of the edge closest to the center of

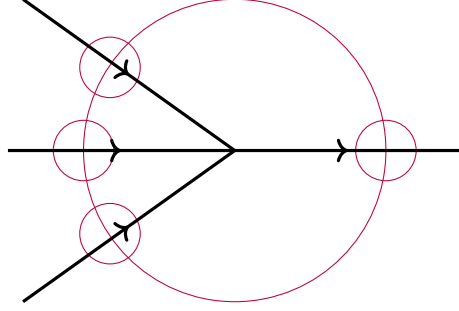


FIGURE 2. The case of a simple junction

the cube. We adapt this to the double case to create a detector for a -junctions, and one for b -junctions, according to the following procedure. Given a specific embedding of a tree inside a cube, we look at a smaller detector cube centered inside the unit cube and we record what it contains:

- if it contains a single a -junction and no other connected component of the tree, we record the position of the a -junction, which is a point inside the detector cube, and rescale it to a point of the unit cube.
- if it contains anything else, we record "nothing", which we set as the base-point.

Therefore, the range of the detector map is a unit cube $[0, 1]^N$, whose boundary has been collapsed to a point, which we call the basepoint.

The detector map that we define in the program is actually a composition of `make_geometric_tree_at` and the aforementioned procedure:

- Take a point in the cube $[0, 1]^{N+1}$ that we can write (x, t) where $x \in [0, 1]^N, t \in [0, 1]$;
- Construct an embedding of a tree inside the cube $[0, 1]^N$. The barycenter of the junctions is at x and the shape of the tree is determined by t ;
- Look inside a smaller detector cube and record what you see;
- Get a point in $([0, 1]^N)/(\partial[0, 1]^N)$.

This is what the functions `detect_a` and `detect_b` correspond to.

4. DEGREE OF A MAP BETWEEN SPHERES

The detector map is defined from $[0, 1]^{N+1}$ to $[0, 1]^N/\partial[0, 1]^N$. If we restrict the source to the boundary of $[0, 1]^{N+1}$, then the source is equivalent to an N -sphere. The range $[0, 1]^N/\partial[0, 1]^N$ is a cube whose boundary is collapsed to a point, which is also equivalent to an N -sphere, as illustrated in figure 3. Therefore, the detector map is equivalent to a function from a sphere S^N to itself.

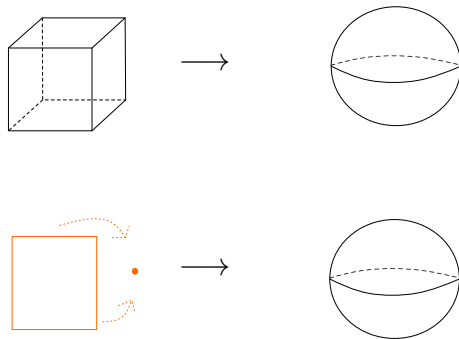


FIGURE 3. The boundary of the 3-dimensional cube and a 2-square whose boundary is glued onto a point are both models for the 2-sphere.

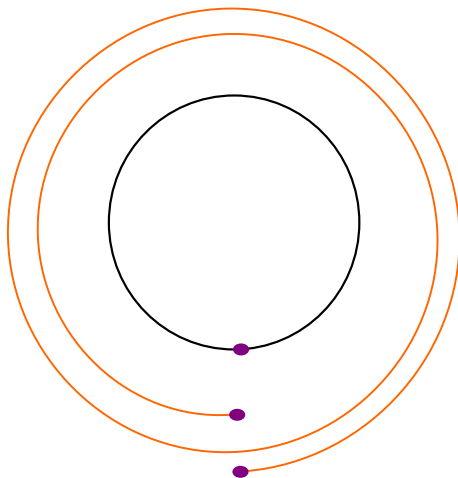


FIGURE 4. The orange circle wraps twice around the first one, starting and ending at the purple point.

The notion of degree we defined in section 2 is closely related to the notion of degree of maps between spheres. Basically, the degree of a map from a sphere S^N to itself is the number of times the map goes around the target sphere, up to homotopy. It is clearer to see this on an example.

Example 4.1. Imagine the function from the circle S^1 to another S^1 which wraps the circle around itself twice, as illustrated in figure 4: The degree of this map is 2.

Examples on spheres of higher dimension are more difficult to understand than in dimension 1, but they work in the same way.

4.1. Degree formula. If the map is C^1 , we can obtain the degree by picking a regular point y in the image (i.e. a point with nonsingular differential), and counting

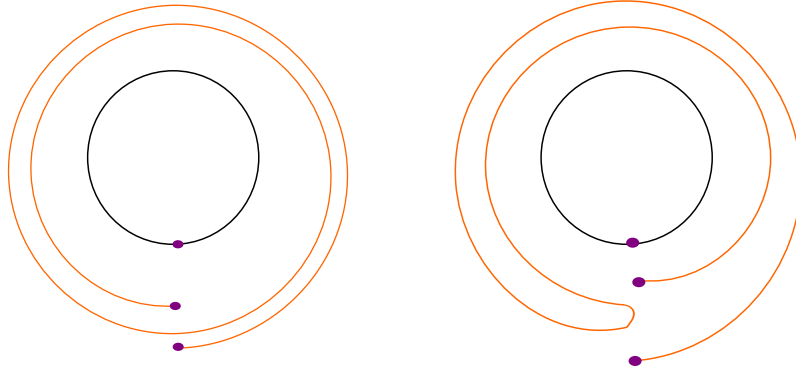


FIGURE 5. The left map is the one introduced earlier. The map on the right wraps the orange circle around the black circle once, then changes directions and wraps around it once more.

its preimages:

$$d = \sum_{p \in f^{-1}(y)} s_p$$

where s_p is the sign of the Jacobian of f at point p .

Example 4.2. Consider the two maps from S^1 to S^1 depicted in figure 5: If one were simply to count the preimages at any regular value – for instance, not at the basepoint of the circle (the purple point), one would see that for both maps, there are two points of the orange circle mapped to the same point of the black circle. However, the degree of the left map is 2, and the degree of the right map is 0. This is because in the first instance, both preimages contribute the same sign to the formula: they are oriented in the same way. In the second instance, they have opposite orientation.

In the program, there is a function `degree_on_cube_map` which computes the degree of any map from a cube to a cube which sends the boundary to the boundary. The function `tree_detector_boundary_degree` computes the degree of the a - and b -detectors.

4.2. Algorithm. Let us now go through the steps of the algorithm:

- (1) Pick a regular value y (more on this in the next section).
- (2) Discretize the domain $[0, 1]^N$ with a regular grid.
- (3) Scan each cell to identify those which are likely to contain a preimage of y .
- (4) Perform a Newton iteration on the retained cells to get an approximate preimage of y
- (5) Compute the Jacobian at the preimage and get the sign of its determinant.

(6) Add up the signs of all found preimages.

Because Newton's method is a bit expensive, we want to have a good test to determine if a cell is likely to contain a preimage of y . We perform a bounding box test in step 3: for the cell C with set of corners V , we consider the box $\prod_{x \in V} [\min f(x)_i, \max f(x)_i]$. If y is not in this box, we discard the cell as unlikely to contain a preimage of y . This works when the detector map is reasonable.

Picking a regular value for step 1: It is very important to pick a good regular value to carry out the above procedure. For maps with a clear analytic formula, this is easy to determine. We therefore look at the outputs of the detector map, and pick a regular value that lives inside a cluster of other regular values, to make sure it is in a very regular zone.

5. RESULTS

In the simple junction case, we can recover the fact that the degree of the a -junction is $a - 1$. In the double junction case, the degree of the a -detector is $-(b - 1)$ and the degree of the b -detector is $a - 1$.

6. LIMITATIONS

6.1. Limitations of the degree computation. The most time-consuming process is definitely the degree computation, as its complexity is in $O(s^N \times N)$ where s is the number of points in the grid. When $N \geq 5$, only extremely simple computations work. Some possible fixes include doing fewer Newton iterations or computing the Jacobian in a coarser way, but the best idea would probably be to create a grid which is not uniform. We could start with a coarse grid, and mark the cells which are likely to contain a preimage of our value, only subdivide those cells with a finer grid, and repeat the process until we get a few candidate cells. We could also use more sophisticated programs to find roots.

6.2. Other problems. The parameters `junction_span_a`, `junction_span_b`, `junction_time_gap` and `detector_radius` are crucial to set correctly. This is because the actual detector map constructs an infinitesimal detector cube, but as we cannot do that we need to approximate it by a small enough cube. The values we have defined work here but they would not work for larger values of a and b . We could set a formula for these parameters which depends on a and b .

We also note that the model for trees is not that readily adaptable to trees with triple junctions, i.e. the case of trees with a -junctions, b -junctions, and c -junctions, which are allowed to collide and switch pairwise.